

\$ KUBECTL APPLY

Kubernetes 101

Capítulo 2 Arquitectura de Kubernetes

El capítulo completo, tal y como aparece en el libro. Los ocho diagramas incluidos.

REGALO PARA SUSCRIPTORES

Javier Vela Aylón

www.javivela.dev · [@jvela](https://twitter.com/jvela)

Arquitectura de Kubernetes

La arquitectura de Kubernetes permite orquestar y ejecutar contenedores a escala de una manera distribuida.

Para ello emplea un conjunto de componentes que interactúan entre sí a través de una API para mantener el estado definido por los usuarios.

Estos componentes se agrupan en dos grandes bloques:

- El control plane, que toma las decisiones globales sobre el clúster: qué Pods se ejecutan, dónde se ejecutan y cómo reaccionar cuando algo cambia respecto al estado deseado.
- Los nodos, las máquinas (físicas o virtuales) donde realmente se ejecutan los Pods.

En clústeres desplegados con herramientas como `kubeadm`²², los propios componentes del control plane se ejecutan como Pods sobre los nodos de control plane. Por eso, estos nodos también necesitan ejecutar `kubelet`²³, aunque no alojen cargas de trabajo de usuario.

A continuación, explicaremos cada uno de estos componentes en detalle.

²²kubeadm. Herramienta oficial para instalar y arrancar un clúster de Kubernetes conforme a las buenas prácticas. Despliega los componentes del control plane como Pods sobre los nodos de control.

²³kubelet. Agente que se ejecuta en cada nodo. Recibe del API Server los PodSpec que le corresponden, pide al runtime que arranque los contenedores y reporta el estado del nodo y de los Pods.

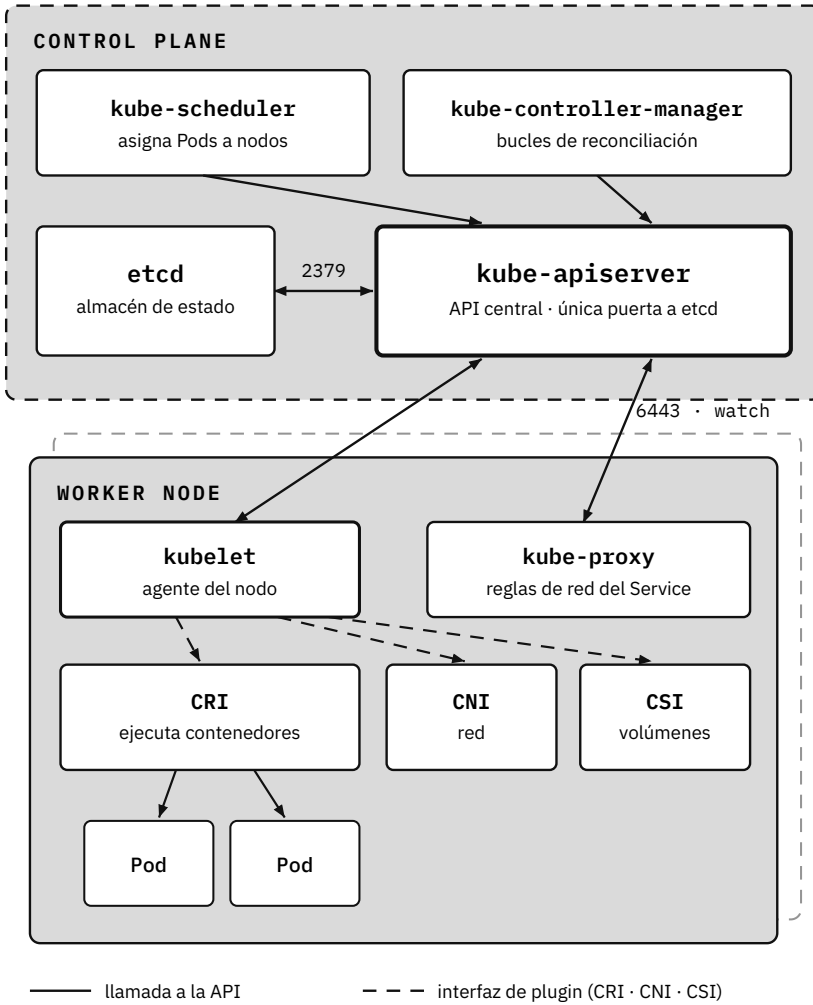


Figura 4: Arquitectura del clúster: control plane y nodos

Fíjate en que todas las flechas pasan por el kube-apiserver: ningún componente habla directamente con otro, ni siquiera los que viven en la misma máquina. Eso es lo que hace que solo haya un punto que proteger de verdad, y también que solo haya uno que se pueda saturar.

Componentes clave

etcd

En todo momento Kubernetes monitoriza, analiza y realiza las operaciones necesarias para poder mantener el estado deseado por los usuarios.

Para almacenar esta información hace uso del almacén de clave-valor distribuido `etcd`.

Al estar distribuido en varias instancias, `etcd` ofrece alta disponibilidad: si una instancia falla, el resto puede seguir respondiendo peticiones sin interrupción. Esto es distinto de recuperarse de un fallo catastrófico de todo el clúster, que depende de disponer de backups de `etcd` (lo vemos un poco más abajo).

Las principales características de `etcd`:

- **Consistencia:** utiliza el algoritmo Raft²⁴, garantizando que los nodos tengan la misma versión de los datos en todo momento.
- **Alta disponibilidad:** los datos se pueden replicar en múltiples instancias, lo que permite seguir accediendo a los datos en caso de que una instancia falle.
- **Interfaz simple:** desde la versión 3, `etcd` expone su API de forma nativa mediante gRPC²⁵, con operaciones CRUD (create, read, update, delete). También existe un gateway que permite acceder a esta misma API a través de HTTP/JSON.
- **Watch API:** permite suscribirse a cambios en claves específicas para obtener notificaciones cuando se produce un cambio en los valores de la clave.

`etcd` es una parte crítica de Kubernetes debido a algunas de sus limitaciones que conviene tener en cuenta:

- **Latencia:** una alta latencia puede provocar timeouts y cambios frecuentes en el líder (leader).
- **Acceso a disco:** es recomendable la utilización de discos SSD. El rendimiento de `etcd` está muy relacionado con el rendimiento del disco utilizado.

²⁴Raft. Algoritmo de consenso distribuido que utiliza `etcd` para elegir un líder y garantizar que todas las réplicas mantienen los mismos datos. Explicación interactiva: <https://raft.github.io/>

²⁵gRPC. Protocolo de llamada a procedimiento remoto (RPC) sobre HTTP/2 que serializa los mensajes en formato binario (Protocol Buffers). Es más eficiente que REST/JSON y admite streaming bidireccional.

- **Limitaciones de tamaño:** el tamaño aproximado de un único valor (value) es de ~1 MiB. Por defecto, el tamaño máximo del almacén es de 2 GB, ampliable hasta un máximo recomendado de 8 GB. El tiempo de arranque es proporcional al tamaño de los datos. A mayor tamaño, mayor tiempo de arranque.

Para finalizar, y en caso de que no utilices un clúster de un cloud provider, debes aplicar las siguientes prácticas:

Backups: realizar backups frecuentes permite restaurar el estado del clúster en caso de tener un fallo catastrófico. `etcdctl` puede ayudarte en esta tarea.

Monitorización: tener configurada una correcta monitorización del uso de CPU, memoria y espacio en disco nos ayuda a detectar de manera proactiva un posible problema.

Alta disponibilidad: utilizar varias instancias para ejecutar el servicio evita que tengamos un punto único de fallo.

`etcd` es un proyecto que forma parte de la Cloud Native Computing Foundation (CNCF - <https://www.cncf.io/>), puedes leer más sobre él en su página web <https://etcd.io/>.

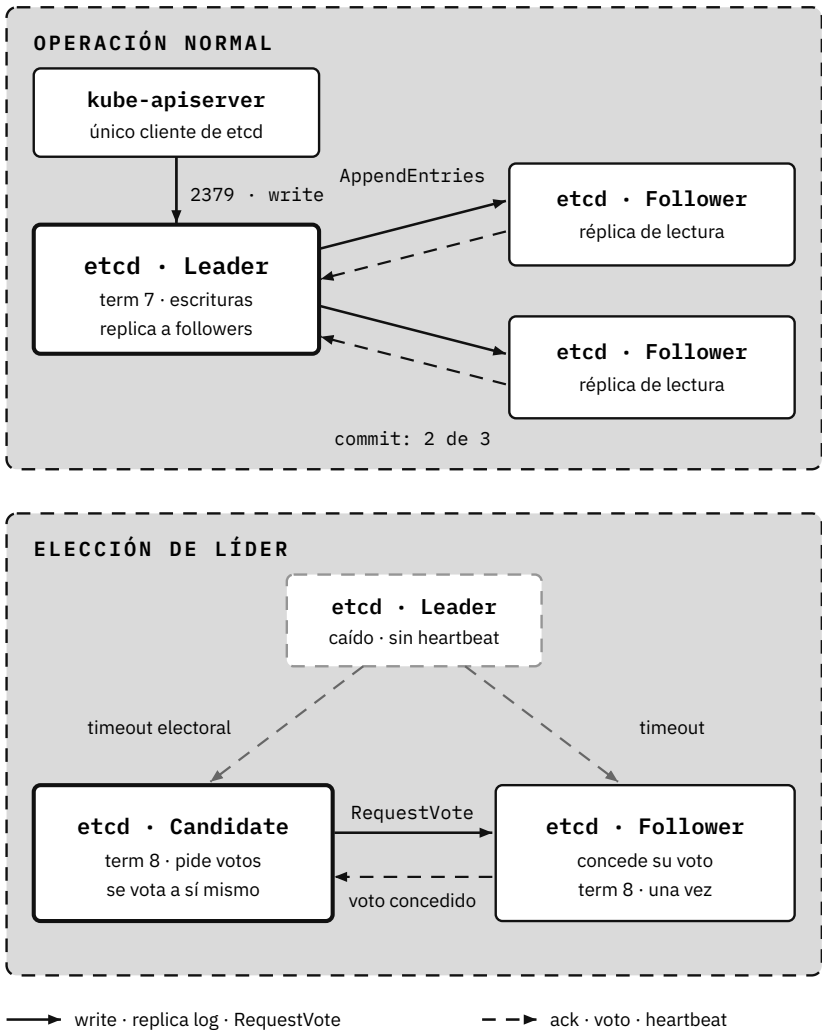


Figura 5: etcd: elección de líder con Raft

Una escritura se confirma cuando la acepta la mayoría, no todos: en un clúster de tres, con dos basta. Por eso etcd se despliega siempre en número impar, y por eso perder dos de tres miembros deja el clúster en solo lectura.

kube-apiserver

Kubernetes API server es la API donde se reciben todas las peticiones del resto de componentes del clúster y los usuarios del clúster tanto para crear nuevos elementos como para consultar su estado.

El protocolo REST nos permite interactuar con este único punto de comunicación con el clúster de Kubernetes. También se encarga de almacenar o actualizar la información necesaria en la base de datos etcd.

Antes de ejecutar una petición, esta es autenticada, autorizada y analizada por los admission controllers. En las siguientes páginas describiremos cada uno de estos elementos.

Por defecto, el puerto utilizado por la API es el 6443 y está protegido por TLS (Transport Layer Security).

Es posible configurar más de un método de autenticación. Al menos deben existir dos métodos:

- Uno para autenticar las peticiones que provienen de las Service Accounts (SA)
- Otro para autenticar a los usuarios del clúster

En el caso de que haya más de un módulo de autenticación activado, se van ejecutando secuencialmente hasta que uno de ellos puede validar correctamente la petición.

Los módulos de autenticación y autorización se explican en el capítulo de Seguridad.

Admission controller

Además de los módulos de autenticación y autorización, el kube-apiserver ejecuta una cadena de admission controllers (siempre hay varios activos) que comprueban todas las peticiones después de autenticarlas y autorizarlas.

Su misión es comprobar el contenido de cada petición para validarlo y modificarlo según se necesite.

La mayoría de admission controllers vienen integrados en el propio binario del kube-apiserver (por ejemplo, NamespaceLifecycle, LimitRanger o ResourceQuota). Además, existen dos tipos de admission controllers extensibles, que delegan la decisión en un servicio externo mediante webhooks:

- **Mutating webhooks:** modifican el contenido de la petición realizada. Por ejemplo, pueden añadir etiquetas (labels) específicas a un Namespace.

- **Validating webhooks:** verifican el contenido de cada una de las peticiones realizadas. Por ejemplo, pueden comprobar que todos los Pods creados tengan configuradas unas propiedades específicas.

Además de los webhooks, Kubernetes permite escribir políticas de admisión sin desplegar ningún servidor externo, usando expresiones CEL (Common Expression Language) declaradas directamente en un objeto de la API. Las de validación (`ValidatingAdmissionPolicy`) son estables desde la versión 1.30, y las de mutación (`MutatingAdmissionPolicy`) desde la 1.36. La ventaja frente a un webhook es que no hay un servicio que mantener, que actualizar ni que pueda caerse: si tu webhook deja de responder y su `failurePolicy` es `Fail`, todas las peticiones afectadas se bloquean, y ese es un modo de fallo que las políticas CEL simplemente no tienen.

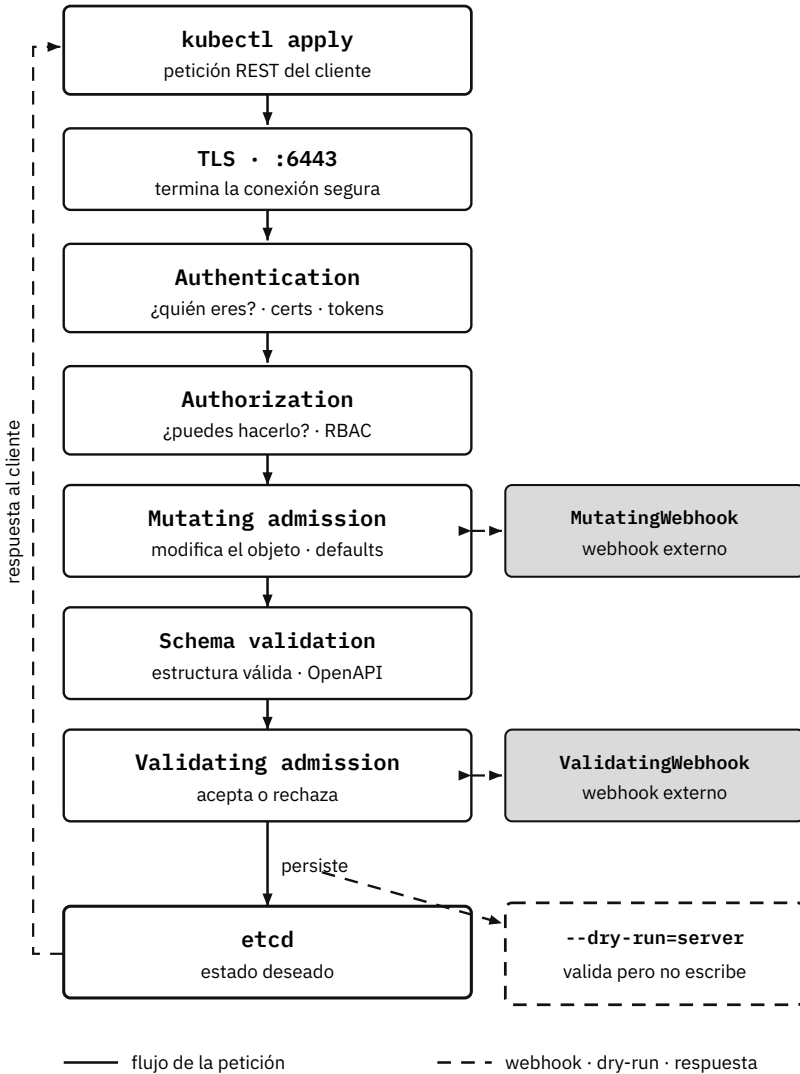


Figura 6: El ciclo de una petición en el kube-apiserver

Las tres etapas se evalúan siempre en este orden y responden preguntas distintas: quién eres, qué puedes hacer y si lo que pides es aceptable. Saber cuál te ha rechazado ahorra medio diagnóstico: un `Unauthorized` viene de la primera, un `Forbidden` de la segunda y casi cualquier otro mensaje raro, de la tercera.

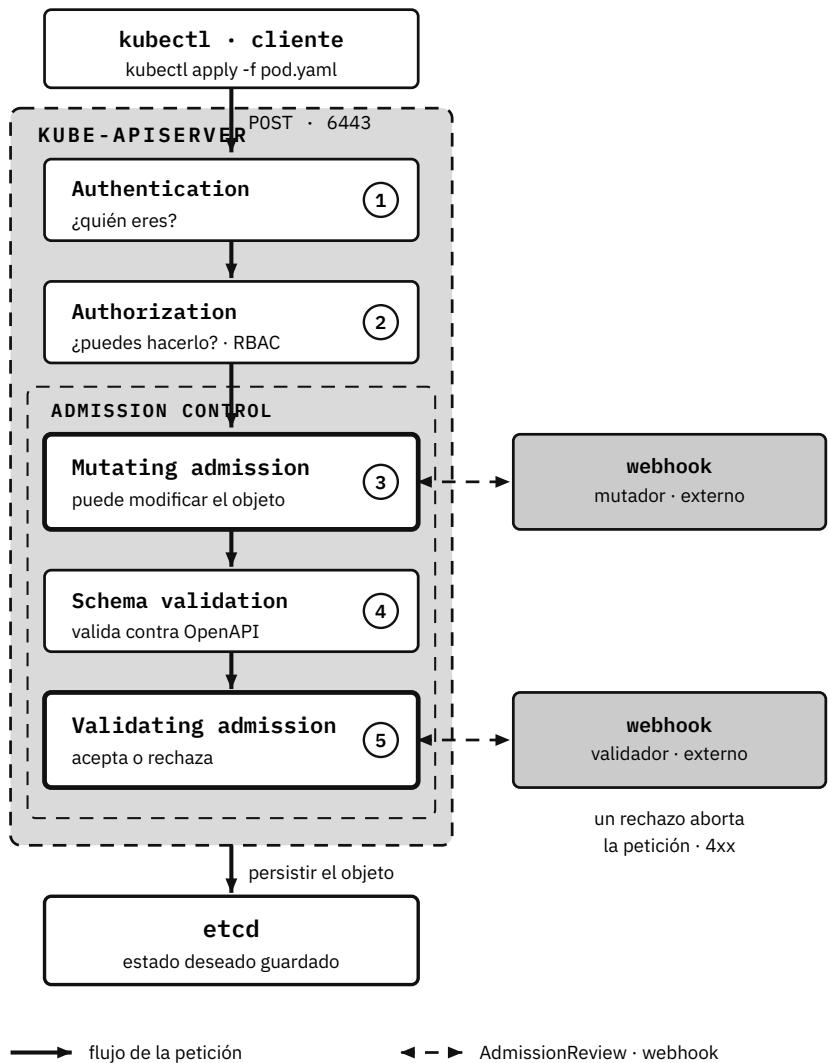


Figura 7: Pipeline de admission controllers: mutación y validación

Los Mutating webhooks se ejecutan antes que los validadores: cuando un validador ve el objeto, ya no cambiará más. Los Validating webhooks pueden llamarse en paralelo; cualquier rechazo aborta toda la petición. `--dry-run` recorre todo el pipeline (admisión incluida) pero no se guarda en etcd.

kube-scheduler

Este componente se encarga de asignar Pods a los nodos más adecuados del clúster. Monitoriza constantemente los Pods que no tienen un nodo asignado y realiza dos operaciones principales:

- **Filtering:** Genera una lista inicial de nodos candidatos donde el Pod podría ser ejecutado. Este proceso tiene en cuenta los recursos necesarios para ejecutar el Pod, *taints*, y reglas de *affinity/anti-affinity*.
- **Scoring:** Evalúa cada uno de los nodos candidatos utilizando las *scoring rules* activadas en la configuración.

Finalmente, asigna el Pod al nodo con la mayor puntuación. Si varios nodos empatan en la puntuación máxima, el scheduler elige uno de ellos al azar, con la misma probabilidad para todos. No hay preferencia por orden alfabético ni por el nodo que respondió antes: dos Pods idénticos creados a la vez pueden acabar en nodos distintos, y eso es intencionado.

Si lo necesitas, puedes implementar tu propio kube-scheduler personalizado o extender el existente mediante la creación de plugins específicos.

En 2021 kube-scheduler-simulator nació como proyecto de Google Summer of Code, y en abril de 2025 el blog oficial de Kubernetes le dedicó una entrada presentando sus capacidades: nos permite consultar a través de una interfaz web las diferentes decisiones que se van ejecutando hasta que un Pod es asignado a un nodo.

Puedes encontrar más información en los siguientes enlaces:

- <https://kubernetes.io/blog/2025/04/07/introducing-kube-scheduler-simulator/>
- <https://github.com/kubernetes-sigs/kube-scheduler-simulator>

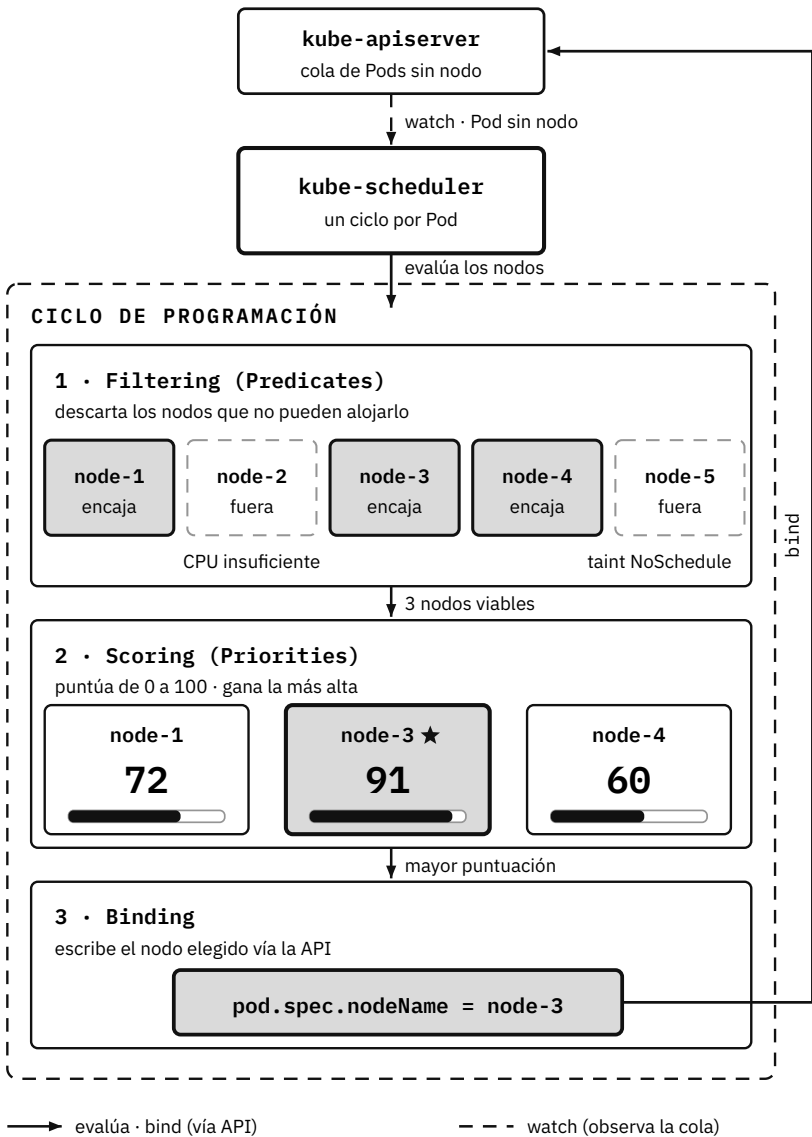


Figura 8: kube-scheduler: filtering, scoring y binding

Si ningún nodo supera el filtrado, el Pod no falla: se queda en Pending y el scheduler lo reintenta. El motivo aparece en los Events del Pod como FailedScheduling.

kube-controller-manager

El kube-controller-manager es un único proceso que ejecuta internamente múltiples controladores (también llamados control loops), cada uno responsable de monitorizar un tipo de recurso y de asegurarse de que se encuentra en el estado correcto.

A continuación se describe una lista con algunos de los controladores que se ejecutan dentro del kube-controller-manager. Más adelante veremos para qué sirve cada uno de los recursos que controlan.

- **Certificate Controller:** Gestiona automáticamente la emisión y renovación de certificados.
- **CronJob Controller:** Ejecuta jobs según una programación temporal definida.
- **DaemonSet Controller:** Asegura que todos los nodos (o algunos filtrados) ejecuten una copia de un Pod específico.
- **Deployment Controller:** Gestiona el despliegue declarativo de aplicaciones y sus actualizaciones progresivas.
- **EndpointSlice Controller:** Crea y actualiza objetos EndpointSlice que conectan Services con Pods.
- **HorizontalPodAutoscaling Controller:** Escala automáticamente el número de Pods basándose en métricas definidas.
- **Job Controller:** Supervisa y gestiona tareas de un solo uso hasta su finalización.
- **Namespace Controller:** Gestiona el ciclo de vida de los Namespaces y su limpieza.
- **Node Controller:** Monitoriza los nodos y responde cuando no están disponibles.
- **ReplicaSet Controller:** Mantiene en ejecución el número de réplicas declarado por cada ReplicaSet.
- **ResourceQuota Controller:** Aplica las restricciones de recursos por Namespace.
- **Service Account Controller:** Crea cuentas de servicio predeterminadas en nuevos Namespaces.
- **StatefulSet Controller:** Gestiona el despliegue y escalado de aplicaciones con estado.

cloud-controller-manager

Cuando el clúster se ejecuta sobre un proveedor cloud (AWS, GCP, Azure...), existe un componente adicional llamado `cloud-controller-manager`. Su función es aislar la lógica específica de cada proveedor del resto del control plane, de forma que el `kube-controller-manager` pueda seguir siendo independiente del proveedor sobre el que se ejecuta.

Entre sus responsabilidades habituales se encuentran:

- **Node Controller:** comprueba en el proveedor cloud si un nodo ha sido eliminado tras dejar de responder.
- **Route Controller:** configura las rutas de red necesarias en la infraestructura del proveedor.
- **Service Controller:** crea, actualiza y elimina los load balancers del proveedor cuando se crean Services de tipo LoadBalancer.

kubelet

`kubelet` se ejecuta como una aplicación en cada uno de los nodos que componen el clúster, incluidos los nodos de control plane en clústeres autogestionados.

Sus responsabilidades principales son:

- Registrar el nodo en el control plane.
- Obtener las especificaciones de los Pods que debe ejecutar.
- Crear, modificar y borrar los contenedores asociados a un Pod, delegando la ejecución real en el container runtime a través de la Container Runtime Interface (CRI).
- Manejar las diferentes probes (liveness, readiness y startup) configuradas para cada uno de los contenedores.
- Informar al control plane sobre el estado de los Pods que se están ejecutando en el nodo.

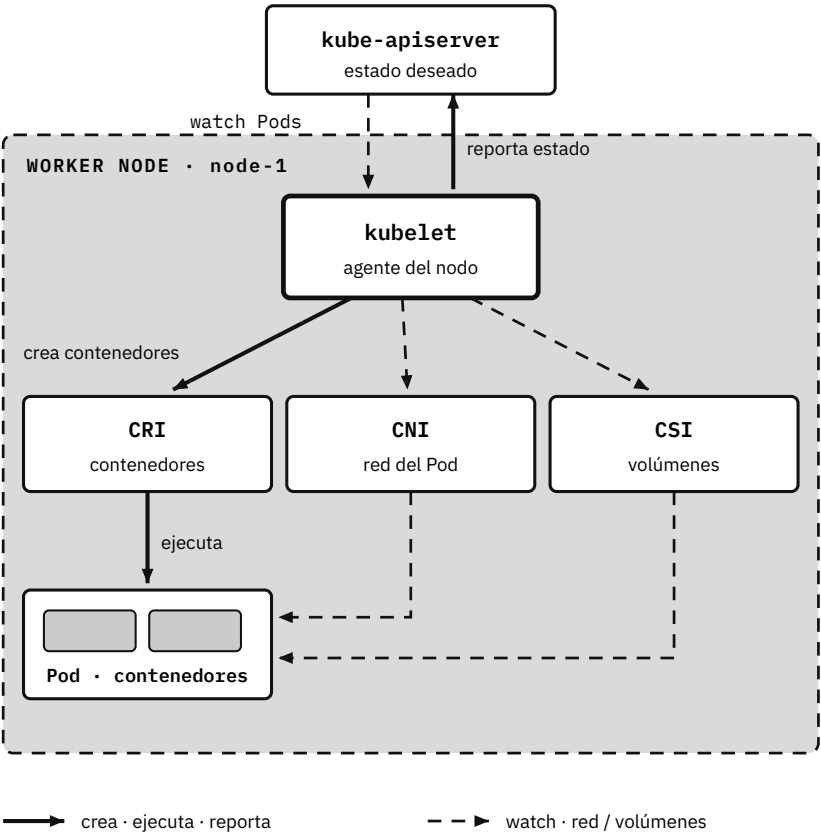


Figura 9: El kubelet y los Pods de su nodo

El kubelet no espera órdenes: vigila el API server y actúa sobre lo que le toca. Por eso un nodo que pierde la conexión con el control plane sigue ejecutando sus Pods tan tranquilo; lo que deja de haber no son contenedores, son cambios.

kube-proxy

kube-proxy se ejecuta en cada uno de los nodos del clúster, habitualmente como un DaemonSet (aunque algunas distribuciones lo despliegan como servicio del sistema o como static Pod). Se encarga de monitorizar los cambios producidos en los Services y EndpointSlices, y de aplicarlos en forma de reglas de red en el nodo en el que se está ejecutando.

Puede operar en varios modos:

- **iptables**: modo histórico y más extendido, basado en reglas de netfilter²⁶.
- **nftables** (estable desde 1.33): modo más reciente, pensado como sucesor de iptables y recomendado para clústeres grandes sobre kernels modernos.

Existe un tercer modo, **IPVS**, pensado para clústeres con un número muy elevado de Services. Se deprecó en la versión 1.35 y será eliminado en próximas versiones²⁷. Si vienes de un clúster antiguo que lo utiliza, migrar a nftables todavía no es obligatorio, pero lo será el día que se retire.

Desde la versión 1.37, si el campo `mode` se deja vacío, kube-proxy lo resuelve como `iptables` y emite un aviso en los logs, porque está previsto que ese valor por defecto cambie a `nftables` en una versión futura. Declararlo de forma explícita es la manera de que una actualización no te cambie el backend de red sin avisar.

²⁶Es el motor interno del kernel de Linux que procesa el tráfico de red y sirve de base para herramientas de seguridad como iptables y nftables.

²⁷La deprecación del modo `ipvs` de kube-proxy y su eliminación se siguen en KEP-5495 (kubernetes/enhancements, issue 5495), que ya publica un calendario en cinco etapas: avisos de deprecación en la 1.35, feature gate `KubeProxyIPVS` en la 1.37, desactivado por defecto en la 1.40, código eliminado del árbol en la 1.43 y limpieza del propio feature gate en la 1.46. En paralelo, KEP-5343 hace de `nftables` el backend por defecto también en la 1.40; desde la 1.37, dejar el campo `mode` vacío emite un aviso. Con esas fechas sobre la mesa, si tu clúster todavía usa `ipvs` tienes hasta la 1.40 para migrar sin que se te desactive solo. Conviene confirmar el calendario en las notas de cada release: las fechas de retirada se han movido antes en otros KEP.

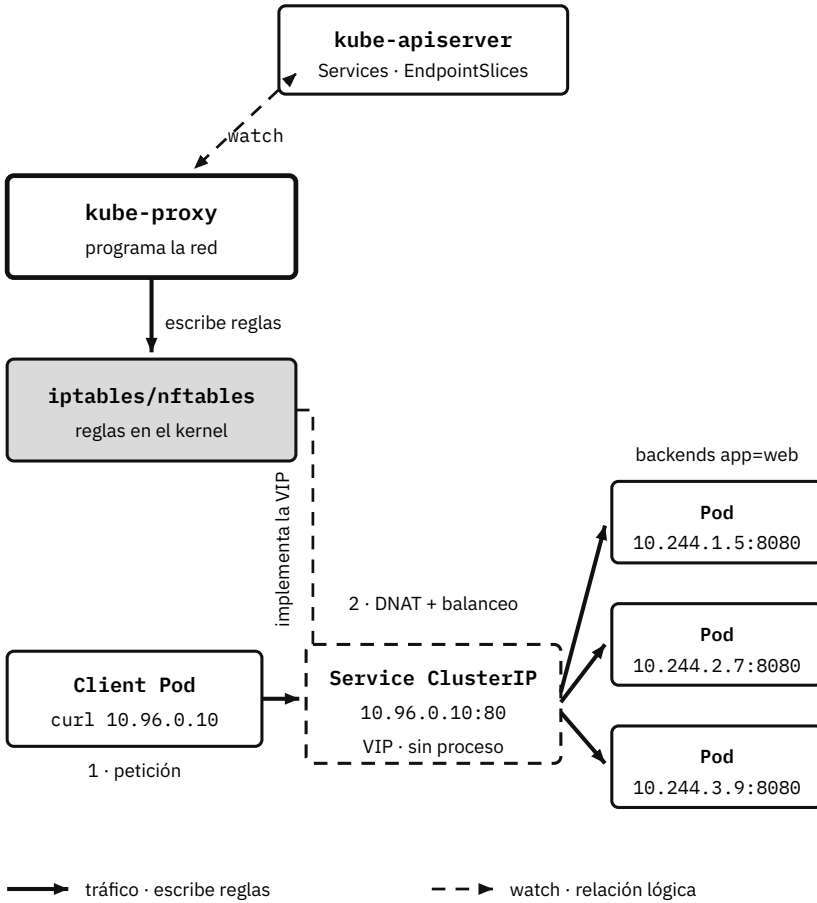


Figura 10: kube-proxy: del Service a las reglas del kernel

kube-proxy no ve pasar el tráfico: programa las reglas y se aparta. Por eso un Service sigue respondiendo aunque kube-proxy esté caído (con las reglas que dejó puestas) y por eso el modo que elijas solo se nota cuando el clúster tiene miles de Services.

Container runtime e interfaz CRI

Cada nodo necesita un container runtime: el software responsable de descargar imágenes, crearlas y ejecutar contenedores. Kubernetes no implementa esta lógica directamente, sino que se comunica con el runtime a través de la Container Runtime Interface (CRI), una API estándar que permite usar diferentes runtimes sin modificar `kubelet`.

Los runtimes más utilizados actualmente son `containerd` y CRI-O. Cómo se llegó hasta aquí, qué papel jugó `dockershim` y qué permite hacer `RuntimeClass` está contado en la sección “Cómo se ejecutan los contenedores: CRI, runtimes y OCI” del capítulo “Introducción a los contenedores y la orquestación”.

Comunicación entre componentes

Kubernetes implementa el patrón hub-and-spoke, en el que el `kube-apiserver` actúa como único punto central de comunicación. Salvo el propio API Server y la excepción del `kubelet` (que veremos a continuación), ningún componente expone puertos para recibir conexiones remotas de forma directa.

Acceso al API Server

Se accede principalmente mediante HTTPS. Desde dentro del clúster se hace a través del Service `kubernetes`, que escucha en el puerto 443 y redirige al 6443 del API server; desde fuera, directamente al 6443. Soporta múltiples estrategias de autenticación: certificados X.509, tokens de ServiceAccount, OpenID Connect, webhook, etc.

Autenticación de nodos y Pods

Los nodos se autentican con certificados de cliente (generados normalmente por `kubeadm` o herramientas similares).

Los Pods utilizan ServiceAccounts. Kubernetes monta automáticamente el token y el certificado CA en cada Pod, permitiéndole comunicarse de forma segura con el API Server.

Comunicación entre el API Server y `kubelet`

Esta comunicación es bidireccional y se utiliza para:

- Obtener logs de los Pods (`kubectl logs`).

- `kubectl port-forward`.
- `kubectl attach / kubectl exec`.
- Recopilación de métricas y estado del nodo.

Se establece sobre el puerto HTTPS del `kubelet` (10250), el único puerto que un componente distinto del API Server expone para recibir conexiones remotas directas. Por defecto, el API Server no valida el certificado del `kubelet` (configuración `kubelet-certificate-authority` vacía), lo que representa un riesgo de seguridad que debe mitigarse en producción.

Otras conexiones del API Server

El API Server también puede iniciar conexiones hacia:

- Nodos (a través del `kubelet`).
- Pods (usando el `kubelet` como proxy).
- Services (para proxying interno).

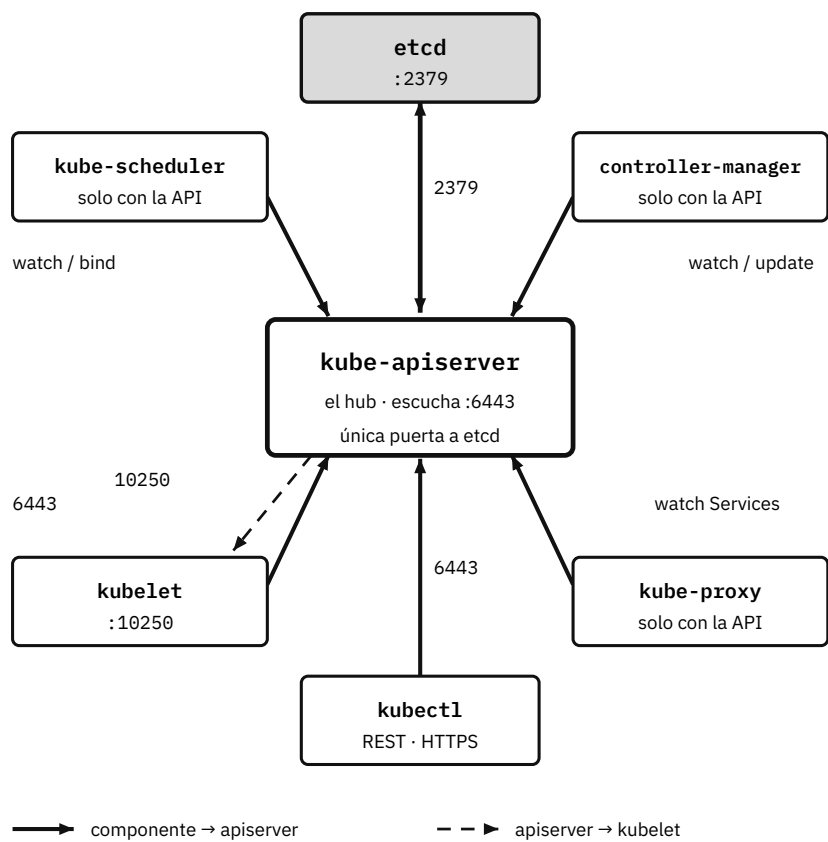


Figura 11: Comunicación entre los componentes del clúster

Ningún componente habla con otro directamente y solo el API Server toca etcd. Por eso un cortafuegos entre nodos apenas necesita abrir dos puertos: 6443 hacia el control plane y 10250 hacia los kubelets.

HASTA AQUÍ EL REGALO

Quedan doce capítulos

Acabas de leer uno de los trece capítulos de **Kubernetes 101**: la guía en español para entender Kubernetes desde cero.

- **13 capítulos**

De qué es un contenedor a observabilidad.

- **3 apéndices**

Glosario, el sistema completo y el estado de cada funcionalidad por versión.

- **48 laboratorios**

Un clúster real en tu navegador. Sin instalar nada.

- **5 retos**

Te entregan un clúster roto. Tú lo arreglas.

Disponible el 1 de septiembre

Kindle 12,99 EUR · Papel 24,99 EUR

Reservar en Amazon

www.javivela.dev/libros/kubernetes-101

(c) 2026 Javier Vela Aylon · Extracto de Kubernetes 101 · Uso personal